Homotopical Encrypt: Hidden Hodge Operators and the Harmonic Cochain Problem

Chloe van der Vlugt

Abstract

We introduce the Harmonic Cochain Problem (HCP), a hidden-operator recovery problem in which public handles, shadows, commitments, or class-level data do not determine the operative Hodge–Laplacian kernel. The underlying cochain decomposition has the form $b = h+e$, where $h \in \ker(\Delta)$ is a harmonic cochain and $e \in \mathrm{im}(\delta) \oplus \mathrm{im}(\delta^*)$ is Hodge-structured noise. We compare HCP with the closest vector problem (CVP): CVP is the known-lattice comparison problem, while HCP hides the operator and kernel that make the relevant lattice explicit. We also describe a certified projection chain from accepted hidden/geometric state to a finite Reed–Solomon mutual-correlated-agreement target, with theorem boundaries formalized in Lean.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Contributions . . . 3

2 The Harmonic Cochain Problem **4**
- 2.1 Public And Hidden Data . . . 6
- 2.2 Recovery Target . . . 8
- 2.3 Computational Model . . . 8

3 CVP Comparison And Kernel Ambiguity **8**
- 3.1 Known-Lattice CVP Surface . . . 9
- 3.2 Reduction Direction . . . 9
- 3.3 Oracle Insufficiency . . . 11
- 3.4 Structural Comparison . . . 13

4 Axioms, Semantics, And Public Data **13**
4.1 Fibrational Semantics . . . 13
4.2 Fibrational Identity Objects . . . 14
4.3 Homotopy Classes . . . 14
4.4 Axiom Surfaces . . . 15
4.5 Exact Structural Invariants . . . 15
4.6 Spectral-Gap Admissibility Surface . . . 16
4.7 Heyting And Oracle Judgment Surface . . . 21

5 Public Artifact Stack And Lossy Projection **22**
5.1 Public, Artifact, And Private Layers . . . 22
5.2 The Artifact Stack . . . 23
5.3 Cochain Commitments . . . 23
5.4 Commitment Map . . . 24
5.5 Commitment And Artifact Invariants . . . 24
5.6 Sheaf Storage And Witness Material . . . 25
5.7 Lossy Projection . . . 25
5.8 Preserved And Lost Structure . . . 25
5.9 Fundamental Properties . . . 26

6 Reed–Solomon Projection And MCA **26**
6.1 Projection Chain . . . 27
6.2 Projection-Stage Gates . . . 27
6.3 Canonical Reed–Solomon Endpoint . . . 28
6.4 Mutual Correlated Agreement . . . 29
6.5 Corollary . . . 30

7 Formal Verification **30**
7.1 Lean Formalization . . . 30
7.2 Repository Structure . . . 30
7.3 Theorem Correspondence . . . 31

8 Future Directions **33**
8.1 Cryptographic Protocol . . . 33
8.2 Anonymous Identity System . . . 33
8.3 Sheaf Storage . . . 33
8.4 Vertra Chain . . . 34
8.5 Complexity-Theoretic Extensions . . . 34

A Repository Dependency Graph **34**

B Lean/Paper Correspondence **35**
B.1 Definitions And Interfaces 35
B.2 Theorems And Status 36

C Complete Lean Coverage Ledger **38**
C.1 Build And Aggregate Files 38
C.2 HCP, CVP, And Fibrational Algebra 38
C.3 Foundational, Sheaf, Spectral, And Judgment Axioms 39
C.4 Laplacian Geometry And Search Spaces 41
C.5 Commitments, Public Artifacts, And TCB Boundary 41
C.6 Projection Chain, Proximity Gates, And RS/MCA Endpoint 42
C.7 Sequel-Facing Protocol And Implementation Boundary 43

1 Introduction

1.1 Motivation

Homotopical Encrypt is organized around the following security shape:

$$\begin{array}{c}\text{public handles}\\ \text{and verifier-facing artifacts}\end{array} \not\Rightarrow \text{the operative Hodge–Laplacian kernel.}$$

The private recovery object is the harmonic cochain selected by a hidden Hodge operator, instead of a scalar or a vector secret in a published lattice or algebraic object. Public verification therefore needs finite artifacts, while security must not collapse to recovering the private representative or the hidden operator.

1.2 Contributions

This paper develops the following contributions.

(i) The Harmonic Cochain Problem (HCP), a hidden-operator recovery problem for Hodge-theoretic cochains.

(ii) A Hodge-structured noise model with cochain decomposition $b = h + e$, where $h \in \ker(\Delta)$ and $e \in \mathrm{im}(\delta) \oplus \mathrm{im}(\delta^*)$.

(iii) A comparison with CVP, separating known-lattice closest-vector search from hidden-operator harmonic-cochain recovery.

(iv) A projection framework from hidden/geometric state to public finite evidence.

(v) A Lean formalization of the definitions, reductions, projection chain, gates, and theorem boundaries.

(vi) A Reed–Solomon mutual-correlated-agreement corollary aligned with the vocabulary of the Proximity Prize problem statement.

2 The Harmonic Cochain Problem

This section states the central search problem and the hidden-operator model in which it is used.

Definition 2.1 (Harmonic Cochain Problem)**.** *The* Harmonic Cochain Problem *(HCP) asks, given public cochain/class-level data associated to a Hodge system, to recover the harmonic cochain determined by the operative Laplacian kernel.*

In the Homotopical Encrypt public-sample model, the visible cochain has the form

$$b = h + e, \qquad h \in \ker(\Delta), \qquad e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*).$$

Here h is the harmonic cochain, equivalently the global-section component, and e is Hodge-structured noise. The hidden-operator condition is that the public data does not determine a unique Δ or $\ker(\Delta)$.

Definition 2.2 (Matrix harmonicity)**.** *For a hidden linear Hodge operator Δ, a cochain $h \in R^N$ is Δ-harmonic when*

$$\Delta h = 0.$$

Thus the harmonic kernel is $\ker(\Delta)$*. In the projection model this agrees with the condition $h \in W$ when $\Delta = \Delta_W = I - P_W$.*

Definition 2.3 (Projection Laplacian, harmonic projection, and residual)**.** *Let $V = R^n$ be equipped with its standard inner product, and let $W \subset V$ be a linear subspace. Write*

$$P_W : V \to W \subset V \qquad \textit{and} \qquad P_{W^\perp} : V \to W^\perp \subset V$$

for the orthogonal projections. The projection-model Laplacian attached to W is

$$\Delta_W = I - P_W.$$

For $b \in V$, define the harmonic projection, residual vector, and residual score by

$$\pi_W(b) = P_W b, \qquad r_W(b) = \Delta_W b, \qquad \rho_W(b) = \|r_W(b)\|.$$

We say that b is W-harmonic when $r_W(b) = 0$.

Proposition 2.4 (Projection harmonicity equivalences)**.** *For every $b \in V$,*

$$r_W(b) = P_{W^\perp} b, \qquad \rho_W(b) = \|P_{W^\perp} b\|, \qquad \rho_W(b) \geq 0.$$

Moreover

$$\pi_W(b) \in W, \qquad r_W(\pi_W(b)) = 0,$$

and the following conditions are equivalent:

$$r_W(b) = 0 \quad \Longleftrightarrow \quad \rho_W(b) = 0 \quad \Longleftrightarrow \quad b \in W.$$

Proof. Since $V = W \oplus W^\perp$ orthogonally, every $b \in V$ decomposes uniquely as

$$b = P_W b + P_{W^\perp} b.$$

Hence

$$(I - P_W)b = b - P_W b = P_{W^\perp} b,$$

giving $r_W(b) = P_{W^\perp} b$ and the displayed formula for $\rho_W(b)$. Nonnegativity is the nonnegativity of the norm. The projection $P_W b$ lies in W, so its orthogonal component is zero; therefore $r_W(\pi_W(b)) = 0$. Finally, $r_W(b) = 0$ iff $P_{W^\perp} b = 0$, which holds iff $b \in W$. The equivalence with $\rho_W(b) = 0$ follows from $\|x\| = 0$ iff $x = 0$. □

Proposition 2.5 (Hodge decomposition surface)**.** *Let*

$$d_{\text{next}} d_{\text{prev}} = 0, \qquad \Delta = d_{\text{prev}} d_{\text{prev}}^* + d_{\text{next}}^* d_{\text{next}}.$$

Then every cochain b admits a decomposition

$$b = h + e_1 + e_2$$

with

$$\Delta h = 0, \qquad e_1 \in \text{im}(d_{\text{prev}}), \qquad e_2 \in \text{im}(d_{\text{next}}^*),$$

and

$$\langle h, e_1 \rangle = \langle h, e_2 \rangle = \langle e_1, e_2 \rangle = 0.$$

Proof. Since Δ is self-adjoint, decompose the ambient cochain space as

$$\ker(\Delta) \oplus \operatorname{im}(\Delta).$$

Since

$$\operatorname{im}(\Delta) \subseteq \operatorname{im}(d_{\text{prev}}) + \operatorname{im}(d^*_{\text{next}}),$$

every cochain b can be written as $h + e_1 + e_2$ with $h \in \ker \Delta$, $e_1 \in \operatorname{im}(d_{\text{prev}})$, and $e_2 \in \operatorname{im}(d^*_{\text{next}})$. From $\Delta h = 0$, one obtains

$$\|d^*_{\text{prev}} h\|^2 + \|d_{\text{next}} h\|^2 = 0,$$

hence $d^*_{\text{prev}} h = 0$ and $d_{\text{next}} h = 0$. These identities give $\langle h, e_1 \rangle = 0$ and $\langle h, e_2 \rangle = 0$, while $d_{\text{next}} d_{\text{prev}} = 0$ gives $\langle e_1, e_2 \rangle = 0$. □

2.1 Public And Hidden Data

Definition 2.6 (Public observation)**.** *A public observation is the data exposed to a verifier or adversary before choosing a hidden representative. In the abstract model it consists of a public class or shadow value p, together with a cochain-facing observation b. In the Homotopical Encrypt instantiation, the fully public handles are hashes, roots, topology hashes, nullifiers, and bounded public summaries; cochain commitments and proof-carrying cochain payloads sit in the verifier-facing artifact layer; full cochains and openings remain witness data unless an explicit protocol path exposes them.*

The public observation is intentionally not the hidden Hodge system. It is a classifier of a public equivalence class of possible hidden representatives. This is the point at which the fibrational and homotopical semantics enter the model: public equality is equality after applying a public classifier, not strict equality of hidden representatives. Section 4 develops this semantic layer.

Definition 2.7 (Hidden-operator public projection)**.** *A public projection is a map*

$$\pi : \{\textit{hidden Hodge operators } \Delta\} \longrightarrow P$$

to a public class type P. Two hidden operators are publicly equivalent when $\pi(\Delta_1) = \pi(\Delta_2)$. The hidden operator fiber over $p \in P$ is

$$\mathsf{Hidden}(p) = \{\Delta : \pi(\Delta) = p\},$$

and the hidden class of Δ is $\mathsf{Hidden}(\pi(\Delta))$.

Definition 2.8 (Lossiness and kernel ambiguity)**.** *The public projection π is* lossy *if there exist $\Delta_1 \neq \Delta_2$ with $\pi(\Delta_1) = \pi(\Delta_2)$. The two operators have the same harmonic kernel if*

$$\forall v, \quad \Delta_1 v = 0 \iff \Delta_2 v = 0.$$

The projection is kernel-ambiguous *if there are publicly equivalent operators Δ_1, Δ_2 whose harmonic kernels are not the same.*

Definition 2.9 (Public-compatible Hodge ambiguity)**.** *Public-compatible Hodge ambiguity means that there exist publicly equivalent operators Δ_1, Δ_2 and a cochain-facing observation b such that*

$$\Delta_1 b = 0 \qquad \textit{but} \qquad \Delta_2 b \neq 0.$$

Proposition 2.10 (Ambiguity hierarchy)**.** *Public-compatible Hodge ambiguity implies kernel ambiguity, and kernel ambiguity implies that the public projection is lossy.*

Proof. If b is harmonic for Δ_1 but not for Δ_2, then the two operators cannot have the same harmonic kernel. This gives kernel ambiguity. If publicly equivalent operators have different kernels, they cannot be the same operator, since equality of operators implies equality of kernels. Therefore the public projection identifies two distinct hidden operators. □

Definition 2.11 (Hidden Hodge witness)**.** *A hidden Hodge witness for a public observation (p, b) consists of hidden operator data Δ, a harmonic cochain h, and Hodge-structured noise e, such that*

$$\Delta \in \mathsf{Hidden}(p), \qquad h \in \ker(\Delta), \qquad e \in \operatorname{im}(\delta) \oplus \operatorname{im}(\delta^*), \qquad b = h + e.$$

Here $\mathsf{Hidden}(p)$ denotes the fiber of hidden Hodge–Laplacian representatives compatible with the public class value p.

Concretely, the hidden data includes the fibration representative, restriction maps, the induced Laplacian, its harmonic kernel, and private witness/cochain data. The public data may bind to this hidden data through commitments or proof-carrying artifacts, but it does not present the operative kernel itself.

Definition 2.12 (HCP solution)**.** *A solution to an HCP instance (p, b) is a cochain h' that is the harmonic component selected by the operative hidden witness. In the exact recovery form used by the hidden-operator separation, $h' = h$ for every hidden witness (Δ, h, e) compatible with the public observation.*

The exact recovery formulation is deliberately strong. It isolates the central ambiguity: the same cochain-facing observation may be compatible with hidden operators that disagree about harmonicity. Thus a solver that sees only public data cannot be uniformly sound unless the public observation determines enough hidden-operator structure.

2.2 Recovery Target

The recovery target is the harmonic cochain selected by the operative kernel, not necessarily the entire kernel subspace. This distinction keeps HCP parallel to CVP: CVP asks for a closest vector in a presented lattice, while HCP asks for a harmonic cochain in the operative Hodge kernel.

In the metric variant used for comparison with CVP, one may ask for the closest harmonic cochain to b. Once Δ or a basis for $\ker(\Delta)$ is explicitly presented, this becomes a closest-vector question on the harmonic lattice. The hidden-operator model withholds precisely that presentation.

2.3 Computational Model

The computational model separates three layers.

(i) The mathematical layer defines public projections, hidden operator fibers, harmonicity, and kernel ambiguity.

(ii) External hardness assumptions and cited coding-theory theorems enter at explicitly labeled theorem boundaries.

(iii) Runtime obligations such as byte parsing, canonical encodings, hash domain separation, proof verification, and state transition are outside the mathematical HCP definition.

This paper focuses on the first two layers. The runtime layer is part of the protocol and proof-carrying-verification story developed in the subsequent protocol paper.

3 CVP Comparison And Kernel Ambiguity

CVP asks for a closest vector in an explicitly presented lattice. HCP asks for a harmonic cochain in the operative Hodge kernel, and in the Homotopical Encrypt model that kernel is not explicitly presented by public data.

3.1 Known-Lattice CVP Surface

An explicit-lattice CVP instance consists of a generator matrix B and a target vector t. The presented lattice is

$$\Lambda_B = \{Bx : x \in R^n\}.$$

A correct CVP answer is a vector $v \in \Lambda_B$ such that

$$\operatorname{dist}(t, v) \leq \operatorname{dist}(t, w) \qquad \text{for every } w \in \Lambda_B.$$

This is the separation between the known-lattice interface and the hidden-operator interface: a CVP oracle receives B and t, while a public HCP procedure sees only the declared handles and verifier-facing artifacts unless extra hidden-operator information is supplied.

3.2 Reduction Direction

The formalization separates three ingredients. The first is pure linear algebra: every presented lattice $\Lambda_B = \operatorname{im}(B)$ can be realized as the kernel of some matrix Δ. The second is the presented-operator Hodge contract: if a solver is sound for the displayed operator Δ, then its output is harmonic for that Δ, hence lies in $\ker(\Delta) = \Lambda_B$. The third is the closestness contract owned by the CVP interface: only after adding minimization among harmonic vectors does the membership statement become a closest-vector statement. Lean keeps these surfaces separate.

Proposition 3.1 (Kernel realization for explicit lattices)**.** *Let B be an explicit generator matrix over a field and set $\Lambda_B = \operatorname{im}(B)$. There exists a matrix Δ such that*

$$\ker(\Delta) = \Lambda_B.$$

Consequently, any solver whose output is Δ-harmonic for every presented Δ returns a vector in the explicit lattice Λ_B when run on this realized operator. If that solver is also closest among Δ-harmonic vectors, then its output is a CVP solution for (B, t).

Proof. Let B be an explicit generator matrix for a lattice

$$\Lambda_B = \{Bx : x \in R^n\} \subseteq R^n$$

and let $t \in R^n$ be the CVP target. We first show the following linear-algebra fact: there is a matrix Δ with

$$\ker(\Delta) = \operatorname{im}(B) = \Lambda_B.$$

Put $V = \mathrm{im}(B)$. Since R^n is finite-dimensional over a field, choose a complementary subspace W with

$$V + W = R^n, \qquad V \cap W = 0.$$

Thus every $y \in R^n$ has a unique decomposition

$$y = v + w, \qquad v \in V, \quad w \in W.$$

Equivalently, there is a unique element $p(y) \in W$ such that $y - p(y) \in V$. Define

$$P(y) = p(y).$$

The uniqueness of the $V \oplus W$ decomposition gives linearity: for $y, z \in R^n$, the vector $P(y) + P(z)$ lies in W, and

$$(y + z) - (P(y) + P(z)) = (y - P(y)) + (z - P(z)) \in V,$$

so $P(y + z) = P(y) + P(z)$. Similarly, for $a \in R$, $aP(y) \in W$ and

$$ay - aP(y) = a(y - P(y)) \in V,$$

so $P(ay) = aP(y)$.

We now compute the kernel. If $y \in V$, then $0 \in W$ and $y - 0 \in V$, hence uniqueness gives $P(y) = 0$. Conversely, if $P(y) = 0$, then $y - P(y) = y \in V$. Therefore

$$\ker P = V = \mathrm{im}(B).$$

Let Δ be the matrix of the linear map P. Then, for every $y \in R^n$,

$$\Delta y = 0 \quad \Longleftrightarrow \quad y \in \ker P \quad \Longleftrightarrow \quad y \in \mathrm{im}(B) \quad \Longleftrightarrow \quad \exists x,\ Bx = y.$$

This is the content of the Lean lemma `exists_matrix_ker_eq_range`.

Now apply a presented-operator Hodge solver to t under this realized operator. In Lean the solver is a function $S : R^n \to R^n$, and the presented-operator soundness hypothesis is

$$\forall \Delta\, b, \qquad \Delta(S(b)) = 0.$$

Applying this with the Δ constructed above and $b = t$ gives

$$\Delta(S(t)) = 0.$$

By the kernel calculation, there exists x with

$$Bx = S(t).$$

Hence $S(t) \in \Lambda_B$. This is exactly the theorem `Hodge_Hardness_Reduction_Proof`: a solver that is harmonic for every presented operator yields a lattice vector for every explicit-lattice instance once the lattice has been realized as a kernel.

Finally, suppose in addition that the solver satisfies the closest-harmonic contract

$$\forall h', \quad \Delta h' = 0 \Longrightarrow \operatorname{dist}(t, S(t)) \leq \operatorname{dist}(t, h').$$

Since the Δ-harmonic vectors are exactly the vectors of Λ_B, this inequality is precisely

$$\operatorname{dist}(t, S(t)) \leq \operatorname{dist}(t, w) \qquad \text{for every } w \in \Lambda_B.$$

Together with $S(t) \in \Lambda_B$, this is the CVP correctness predicate `IsClosestLatticeVector` for the explicit instance (B, t). □

Once the hidden operator data has been fixed or presented, the closest harmonic cochain problem is precisely a CVP-style problem on $\ker(\Delta)$. Thus CVP is a known-lattice shadow of the Hodge recovery problem.

Proposition 3.2 (Projection-model Hodge recovery is CVP-shaped)**.** *Fix a hidden harmonic subspace $W \subseteq R^n$ with orthogonal projection P_W, and let $\Delta_W = I - P_W$. A cochain h is Δ_W-harmonic exactly when $h \in W$. Therefore, if h is harmonic and closest to b among all harmonic cochains, then h is the CVP solution for the target b in the lattice/subspace W.*

Proof. In the projection model,

$$\Delta_W b = (I - P_W)b,$$

equivalently the projection of b to $W^{\perp}$. Hence

$$\Delta_W h = 0 \quad \Longleftrightarrow \quad h \in W.$$

The remaining statement is definitional: a Hodge/CVP solution is precisely a vector that is Δ-harmonic and minimizes $\operatorname{dist}(b, -)$ among all Δ-harmonic vectors. □

3.3 Oracle Insufficiency

The strictness direction uses a different contract. A public Hodge solver sees the cochain-facing observation and may call an explicit-lattice CVP oracle, but it is not given the hidden operator. Soundness asks that the solver's output be harmonic for every operative hidden Δ. Exactness on already-harmonic inputs asks that, if the observation b is already harmonic for the operative Δ, the solver return b unchanged.

Proposition 3.3 (Known-lattice CVP oracle insufficiency)**.** *In the hidden-operator model, an oracle for explicit known-lattice CVP does not by itself determine the operative Hodge–Laplacian kernel. Consequently, such an oracle does not uniformly solve HCP from public data alone.*

Proof. Assume $\dim R^n \geq 2$, and choose an index i. Let

$$b = e_i, \qquad \Delta_1 = 0, \qquad \Delta_2 = \operatorname{diag}(1, 0, \ldots, 0),$$

where the nonzero diagonal entry of Δ_2 is placed at i. Then

$$\Delta_1 b = 0, \qquad \Delta_2 b = b \neq 0.$$

Thus b is harmonic for Δ_1 but not harmonic for Δ_2. If the public projection is lossy enough to identify Δ_1 and Δ_2, the same cochain-facing observation b is compatible with two hidden Hodge operators that disagree about whether b lies in the harmonic kernel.

Now suppose, for contradiction, that there were a public-data-only Hodge solver S which may call a correct explicit-lattice CVP oracle O, and which is uniformly sound and exact on already-harmonic inputs. Since b is harmonic for Δ_1, exactness forces

$$S(O, b) = b.$$

But soundness for the hidden operator Δ_2 forces

$$\Delta_2 S(O, b) = 0.$$

Combining the two equalities gives $\Delta_2 b = 0$, contradicting the construction of b. Hence no such public-data-only solver exists, even assuming the availability of a correct oracle for explicitly presented CVP instances. □

The obstruction is kernel ambiguity: the same public observation may be compatible with multiple hidden operators whose harmonic kernels differ. A CVP oracle can solve a closest-vector problem after the lattice is presented; it does not select which hidden kernel public data was meant to present.

Proposition 3.4 (Concrete public ambiguity)**.** *In every dimension at least two, the constant public projection is public-compatible Hodge ambiguous, hence kernel-ambiguous and lossy.*

Proof. Use the same witnesses as in the proof of Theorem 3.3: $b = e_i$, $\Delta_1 = 0$, and $\Delta_2 = \operatorname{diag}(1, 0, \ldots, 0)$. The constant projection sends every hidden operator to the same public value, so Δ_1 and Δ_2 are publicly equivalent. Since b is harmonic for Δ_1 and not for Δ_2, this gives public-compatible Hodge ambiguity. The implications of Theorem 2.10 then give kernel ambiguity and lossiness. □

3.4 Structural Comparison

The comparison separates three surfaces:

(i) known-lattice closest-vector search;

(ii) hidden-operator HCP;

(iii) public projection/proximity verification.

The first two surfaces are the hardness story. The third is a verifier-facing theorem: given accepted hidden/geometric data, later sections construct finite public shadows and an RS/MCA endpoint without claiming that the projection is injective.

4 Axioms, Semantics, And Public Data

This section fixes the semantic and axiomatic surfaces used by the problem. It comes before the public artifact and commitment layer because those objects are only meaningful after the public/private boundary is fixed.

4.1 Fibrational Semantics

The semantic layer is motivated by Helfer's first-order homotopical logic [Hel19] and by the standard topos-theoretic language of sheaves and internal logic [LM94]. We use it here as a semantic language for hidden representatives, public bases, and transport of witness data; the external foundational results are taken as background rather than reproved. We use the usual fibrational notation

$$p : E \longrightarrow B, \qquad E_b = p^{-1}(b)$$

for a total space of hidden data over a base of public or contextual data. When $x, y \in E_b$, equality in the semantic layer is interpreted through a path object rather than by strict equality in the ambient presentation. We write

$$x \simeq_b y \quad \Longleftrightarrow \quad \mathrm{Path}_{E_b}(x, y) \text{ is inhabited.}$$

For a dependent family $F : E \to \mathsf{Type}$, a path $\gamma \in \mathrm{Path}_E(x, y)$ supplies the transport map

$$\mathsf{transport}_\gamma : F_x \longrightarrow F_y.$$

Thus the primitive semantic data are fibers, paths, and transport along paths. The cryptographic objects introduced later live over this interface; they do not replace it with byte-level equality.

We also use the corresponding categorical fibration vocabulary. For a functor $p : E \to B$, a morphism $f : U \to V$ in E is p-Cartesian when every arrow $g : W \to V$ and base arrow $\alpha : p(W) \to p(U)$ satisfying

$$p(g) = \alpha\, p(f)$$

factor uniquely through f. A Grothendieck fibration is then a functor with Cartesian lifts for every base arrow. This is the categorical form of the transport principle used by the hidden-representative semantics.

Proposition 4.1 (Fibration copies preserve hidden semantics)**.** *Let $p : E \to B$ be a Grothendieck fibration. For each tag n, let $E^{(n)}$ be a tagged copy of E and let $p^{(n)} : E^{(n)} \to B$ be the induced fibration. Then $p^{(n)}$ is again a Grothendieck fibration, and p and $p^{(n)}$ are homotopy equivalent over B.*

Proof. The tagged copy has the same objects and morphisms as E, with only the external tag changed. The forgetful functor $E^{(n)} \to E$ and the tagging functor $E \to E^{(n)}$ compose to the identity up to the evident natural isomorphisms. Cartesian lifts in $E^{(n)}$ are transported directly from Cartesian lifts in E, so the induced functor remains a Grothendieck fibration. The two functors give the homotopy equivalence over the base B. □

4.2 Fibrational Identity Objects

Let $\Delta_A : A \to A \times A$ denote the diagonal. Following the path/diagonal semantics, the identity object over a pair (x, y) has fiber $\mathrm{Path}_A(x, y)$. We therefore regard x and y as equal for the first-order homotopical semantics exactly when this fiber is inhabited:

$$x \simeq_A y \quad \Longleftrightarrow \quad \mathrm{Path}_A(x, y) \neq \varnothing.$$

This is the semantic source of the hidden-operator model used in HCP: the verifier may receive evidence that two representatives lie in the same homotopical identity class without receiving the representative or the path witness itself.

4.3 Homotopy Classes

Let A be a type of hidden representatives and let P be a type of public class data. A public classifier is a map

$$\pi : A \longrightarrow P.$$

It determines public fibers

$$A_p = \pi^{-1}(p) = \{x \in A : \pi(x) = p\}$$

and a public equivalence relation

$$x \equiv_\pi y \quad \Longleftrightarrow \quad \pi(x) = \pi(y).$$

The classifier is path-respecting when

$$x \simeq_A y \Longrightarrow x \equiv_\pi y,$$

and non-faithful when some public fiber contains distinct hidden representatives. A hidden class-membership witness is a path $\gamma \in \mathrm{Path}_A(x, y)$; its public shadow is only the equality $\pi(x) = \pi(y)$. At this stage the shadow is only a classifier or fingerprint. The finite-code projection and Reed–Solomon/MCA endpoint are introduced later in Section 6.

4.4 Axiom Surfaces

The assumptions are separated by mathematical role:

(i) foundational cochain and Hodge assumptions;

(ii) fibration and homotopy-class interfaces;

(iii) Heyting/oracle semantics for public judgments;

(iv) sheaf-coherence and spectral/Tsirelson admissibility interfaces;

(v) cochain-commitment payload and security assumptions.

4.5 Exact Structural Invariants

The exact structural invariants are not a separate source of geometry. They package the same foundational and sheaf-coherence hypotheses at the point where an object is admitted. We use the cellular-sheaf Hodge–Laplacian viewpoint of Hansen–Ghrist as background [HG18]. For a cochain complex $C^\bullet$, the accepted surface supplies a differential

$$d^k : C^k \longrightarrow C^{k+1}$$

with $d^{k+1}d^k = 0$, together with an adjoint d^* and the induced Hodge Laplacian

$$\Delta = d^*d + dd^*.$$

In the sheaf setting, the corresponding coherence assumptions assert that restriction maps, local data, and gluing data are compatible with this cochain/Hodge structure. The only structural consequences used later are the differential identity $d^2 = 0$ and the Hodge–Laplacian identity above.

Proposition 4.2 (Exact structural preconditions)**.** *An accepted sheaf-coherence surface supplies*

$$d^2 = 0 \qquad \textit{and} \qquad \Delta = d^*d + dd^*$$

for the cochain complex under discussion.

4.6 Spectral-Gap Admissibility Surface

The spectral surface is an object-level admissibility interface. It is not the source of HCP hardness, not a reconstruction theorem for the hidden fibration, and not introduced here as a consensus rule. The Lean predicate `SpectralGapInPoVBounds` records the PoV/Tsirelson spectral window used later by the protocol layer. In Paper 1 it is simply a hard mathematical predicate on candidate Hodge–Laplacian data. The terminology is Tsirelson-style because the admissible window uses the familiar $\sqrt{2}$ scale from Tsirelson's bound [Tsi80]. For a Hermitian positive semidefinite operator Δ, write

$$\lambda_1^+(\Delta) = \inf\{\lambda : \lambda \text{ is an eigenvalue of } \Delta,\ \lambda \neq 0\},$$

with $\lambda_1^+(\Delta) = 0$ when Δ has no nonzero spectral value. The Tsirelson-style admissibility axiom is the spectral window

$$\frac{\sqrt{2}}{2} \leq \lambda_1^+(\Delta) \leq 2\sqrt{2}.$$

A separate boundary predicate marks Δ as ε-near one of the two endpoints:

$$\left|\lambda_1^+(\Delta) - \frac{\sqrt{2}}{2}\right| \leq \varepsilon \quad \text{or} \quad \left|\lambda_1^+(\Delta) - 2\sqrt{2}\right| \leq \varepsilon.$$

This predicate only records near-boundary status; runtime policy for such values belongs to the subsequent protocol paper.

The corresponding Grassmannian/Laplacian surface makes the hidden-kernel search space concrete. Let

$$\mathcal{L}_\beta(R^n) = \{\Delta : \Delta = \Delta^*,\ \Delta \succeq 0,\ \dim_R \ker \Delta = \beta\}$$

be the Laplacian space at kernel dimension β. For $W \in \mathrm{Gr}(k, R^n)$, let P_W be the orthogonal projection onto W and set

$$\Delta_W = I - P_W.$$

Thus $\ker(\Delta_W) = W$, and the Grassmannian family is

$$\Omega_k = \{\Delta_W : W \in \mathrm{Gr}(k, R^n)\}.$$

Equivalently, a point of $\mathrm{Gr}(k, R^n)$ is a k-dimensional harmonic subspace W, and the map $W \mapsto \Delta_W$ sends it to the corresponding projection Laplacian. In particular, Ω_1 is the one-dimensional Grassmannian family of rank-one harmonic kernels.

Proposition 4.3 (Grassmannian Laplacian search space)**.** *For every $k \leq n$, the family Ω_k is nonempty and satisfies*

$$\Omega_k \subseteq \{\Delta_W : W \subset R^n \text{ a linear subspace}\}.$$

Moreover, every element of Ω_k has a k-dimensional harmonic kernel, so $\Omega_k \subseteq \mathcal{L}_k(R^n)$.

Proof. If $k \leq n$, choose k coordinate basis vectors in R^n. Their span is a k-dimensional subspace W, and $\Delta_W \in \Omega_k$, proving nonemptiness. The displayed containment is immediate from the definition of Ω_k. Since $\Delta_W = I - P_W = P_{W^\perp}$, its kernel is exactly W, hence has dimension k. □

Proposition 4.4 (Injectivity of the Grassmannian Laplacian map)**.** *If $W_1, W_2 \subset R^n$ are subspaces and*

$$\Delta_{W_1} = \Delta_{W_2},$$

then $W_1 = W_2$. Consequently $W \mapsto \Delta_W$ is injective on $\mathrm{Gr}(k, R^n)$.

Proof. The equality $\Delta_{W_1} = \Delta_{W_2}$ is the equality $I - P_{W_1} = I - P_{W_2}$, hence $P_{W_1} = P_{W_2}$. For any $x \in W_1$, $P_{W_1} x = x$, so $P_{W_2} x = x$, and therefore $x \in W_2$. Thus $W_1 \subseteq W_2$. The reverse containment is identical, so $W_1 = W_2$. □

The higher-rank Grassmannian search spaces contain explicit one-parameter families. Fix $1 \leq k < n$. Choose linearly independent vectors $u, v \in R^n$ and a $(k-1)$-dimensional subspace W_0 disjoint from $\mathrm{span}\{u, v\}$. For $t \in R$, set

$$W(t) = \mathrm{span}\{u + tv\} + W_0.$$

Then $W(t)$ has dimension k, and varying t gives a corresponding family

$$\Delta_{W_t} = I - P_{W_t}.$$

Proposition 4.5 (Parametric Laplacian search space)**.** *If $1 \le k < n$, then there exist u, v, W_0 as above such that the map*

$$R \longrightarrow \mathrm{Gr}(k, R^n), \qquad t \longmapsto W(t)$$

is injective. Hence $t \mapsto \Delta_{W(t)}$ is an injective one-parameter family in Ω_k.

Proof. The existence of u, v, W_0 follows by choosing a $(k-1)$-plane and then two independent directions outside it; the inequality $k < n$ leaves enough room for this choice. The dimension computation uses the disjointness:

$$\dim W(t) = \dim \mathrm{span}\{u + tv\} + \dim W_0 = 1 + (k-1) = k.$$

If $W(t_1) = W(t_2)$, then $u + t_1 v$ lies in $\mathrm{span}\{u + t_2 v\} + W_0$. Modulo W_0, this gives a linear dependence between u and v unless $t_1 = t_2$. Since u, v were chosen independent and W_0 was disjoint from their span, $t_1 = t_2$. Injectivity of $t \mapsto \Delta_{W(t)}$ follows from Theorem 4.4. □

Proposition 4.6 (Projection spectrum and spectral gap)**.** *For every $W \subset R^n$, the projection Laplacian $\Delta_W = I - P_W = P_{W^\perp}$ has spectrum contained in $\{0, 1\}$. Its kernel is W, its image is $W^\perp$, and, if $W \neq R^n$, its least nonzero eigenvalue and largest eigenvalue are both 1. Thus the relative spectral gap of the Grassmannian projection family is 1.*

Proof. If $x \in W$, then $\Delta_W x = 0$. If $x \in W^\perp$, then $\Delta_W x = x$. Since $R^n = W \oplus W^\perp$, these two subspaces exhaust the eigenspace decomposition. Therefore the only possible eigenvalues are 0 and 1, with 0-eigenspace W and 1-eigenspace $W^\perp$. When $W^\perp \neq 0$, the nonzero spectral values consist only of 1, so both λ_1^+ and $\lambda_{\max}$ are 1. □

The likelihood surface records the corresponding adversary vocabulary. A classical adversary is a map from an observation b to a guessed Laplacian; a quantum adversary is a probability density on candidate Laplacians. For a noise scale σ, the Gaussian likelihood of a candidate A at observation b is written abstractly as

$$L_\sigma(b, A) = \exp\left(-\frac{\|Ab\|_2^2}{2\sigma^2}\right),$$

with normalized classical and quantum success probabilities obtained by integrating or maximizing this likelihood over the chosen family Ω_k. These likelihood objects are comparison surfaces for quantitative security statements; they are not assumptions used to prove the hidden-operator separation in Theorem 3.3.

Definition 4.7 (Adversary likelihood surfaces)**.** *A classical adversary on R^n is a function*

$$\mathcal{A}_{\mathrm{cl}} : R^n \to M_n(R)$$

from an observation to a guessed Laplacian. A quantum adversary on a candidate family Ω_k is a nonnegative density

$$\mathcal{A}_{\mathrm{q}} : R^n \times \Omega_k \to R_{\geq 0}$$

whose integral over Ω_k is 1 for each observation. With the standard Gaussian prior on observations, the corresponding normalized success surfaces are obtained from

$$\int_{R^n} L_\sigma(b, \mathcal{A}_{\mathrm{cl}}(b)) e^{-\|b\|_2^2/2}\, db$$

in the classical case and

$$\int_{R^n} \left(\int_{\Omega_k} \mathcal{A}_{\mathrm{q}}(b, A) L_\sigma(b, A)\, dA \right) e^{-\|b\|_2^2/2}\, db$$

in the quantum-density case, up to the usual Gaussian normalization factor.

The likelihood is written with the Euclidean norm. The Lean development also records a formal norm-convention check: the default norm on $n \to R$ is the supremum norm, so the Euclidean likelihood is represented by passing to the ℓ^2 norm explicitly. The mathematical surface used here is the corrected Euclidean one.

Proposition 4.8 (Likelihood for a projection Laplacian)**.** *For $A = \Delta_W$, the Euclidean Gaussian likelihood is*

$$L_\sigma(b, \Delta_W) = \exp\left(-\frac{\|P_{W^\perp} b\|_2^2}{2\sigma^2} \right).$$

Proof. By Theorem 2.4, $\Delta_W b = P_{W^\perp} b$. Substitute this identity into the definition of $L_\sigma(b, A)$. □

Proposition 4.9 (Single-observation optimal classical guess)**.** *If $b \neq 0$ and $W_b = \mathrm{span}\{b\}$, then*

$$L_\sigma(b, \Delta_{W_b}) = 1.$$

Equivalently, the projection residual of b against the Laplacian whose kernel is $\mathrm{span}\{b\}$ is zero.

Proof. Since $b \in W_b$, $\Delta_{W_b} b = 0$. Hence $L_\sigma(b, \Delta_{W_b}) = \exp(0) = 1$. □

This proposition is deliberately local: it concerns one observation b and the candidate subspace generated by that same observation. It does not collapse the HCP search problem, because the hardness comparison ranges over the whole hidden Grassmannian family and over public observations that do not reveal the operative kernel.

Proposition 4.10 (Gaussian prior factorization)**.** *For $x = (x_i)_{i=1}^n \in R^n$,*

$$\exp\left(-\frac{\|x\|_2^2}{2}\right) = \prod_{i=1}^{n} \exp\left(-\frac{x_i^2}{2}\right).$$

Proof. Since $\|x\|_2^2 = \sum_i x_i^2$, the left hand side is

$$\exp\left(\sum_i -\frac{x_i^2}{2}\right),$$

which equals the product of the exponentials. □

Definition 4.11 (Spectral fingerprint)**.** *A finite-dimensional sheaf Laplacian is a self-adjoint positive semidefinite linear operator $L : V \to V$. Its spectral fingerprint is the multiset*

$$\mathrm{Spec}_{\mathrm{multi}}(L)$$

of eigenvalues counted with multiplicity. Two Laplacians are spectrally equivalent when these multisets agree. In contrast, the coarse homotopy equivalence surface used here records only harmonic dimension:

$$L_1 \simeq_{\ker} L_2 \quad \Longleftrightarrow \quad \dim\ker L_1 = \dim\ker L_2.$$

Proposition 4.12 (Spectrum is not determined by harmonic dimension)**.** *There exist self-adjoint positive semidefinite Laplacians L_1, L_2 with*

$$\dim\ker L_1 = \dim\ker L_2$$

but

$$\mathrm{Spec}_{\mathrm{multi}}(L_1) \neq \mathrm{Spec}_{\mathrm{multi}}(L_2).$$

Proof. Take $V = R$, $L_1 = \mathrm{id}_R$, and $L_2 = 2\,\mathrm{id}_R$. Both kernels are zero, so the harmonic dimensions agree. The spectral fingerprints are the singleton multisets $\{1\}$ and $\{2\}$, respectively, so the operators are not spectrally equivalent. □

Proposition 4.13 (Spectral-gap admissibility)**.** *If a candidate Hodge–Laplacian geometry carries a by-construction PoV/Tsirelson admissibility witness, then*

$$\lambda_1^+(\Delta) \in \left[\frac{\sqrt{2}}{2},\, 2\sqrt{2}\right].$$

This is exactly the Lean theorem that the field `gap_in_bounds` of `ByConstructionPoVGeometry` implies the hard spectral predicate, together with the gate-facing alias in `Gates/Spectral.lean`. It constrains which candidate objects are admissible; it does not remove hidden-operator ambiguity. The HCP hardness story in Section 2 remains the hidden fibration, hidden Laplacian, and hidden kernel.

For a nonempty family $\Omega \subseteq \mathcal{L}_1(R^n)$, the associated classical and quantum adversary-bound forms are spectral-statistical targets, not HCP hardness assumptions. The Lean file also contains empty-family sanity-check theorems, which are logically useful boundary checks but are not nontrivial hardness bounds.

4.7 Heyting And Oracle Judgment Surface

Following the Helfer-style fibrational/topos semantics used above [Hel19, LM94], the public judgment layer is a finite Heyting shadow of the richer semantic object carried by hidden representatives. Public truth values are quotients of hidden semantic evidence, not Boolean inspections of the hidden object itself. For the public judgment layer, we use a three-valued Heyting surface

$$\Omega_3 = \{\bot, ?, \top\}, \qquad \bot < ? < \top,$$

with the induced meet, join, and implication operations.

The oracle-specific part is the query/answer mechanism that produces public judgments. Following the oracle-modality viewpoint of Ahman–Bauer [AB26], oracle-visible predicates such as vertex compatibility, Betti-number compatibility, public identity-hash compatibility, and spectral compatibility are mapped into this finite public Heyting surface, and the acceptance predicate is formed in the resulting public logic. The collapse from the richer hidden semantic object to Ω_3 is lossy: distinct hidden witnesses may have the same public truth value. It is also not classical Boolean logic. In the three-chain Heyting algebra, implication is the residuated operation

$$c \le (a \Rightarrow b) \quad \Longleftrightarrow \quad a \wedge c \le b,$$

and the intermediate value ? is not eliminated by classical reasoning. Thus the public oracle layer is a quotient of hidden semantic evidence, not a reconstruction of that evidence.

The intermediate value ? is a logical value in this public algebra, not a Boolean verdict in disguise. Operational maps out of Ω_3, such as protocol actions attached to an intermediate public judgment, belong to the subsequent protocol paper's runtime and consensus semantics. They are not the Heyting implication operation and do not alter the residuation law above.

The identity hash used by the oracle surface is an observation obtained from the public Hodge projection and quantization path. It is not a cochain commitment payload. The corresponding stability assumption says that equal public identity inputs give the same identity hash; it is a determinism condition for public identity observations, not a uniqueness or binding theorem for cochain commitments. Similarly, topological observations such as Betti-number compatibility are homotopy-invariant public data, whereas spectral fingerprints are not determined by homotopy class alone by Theorem 4.12. The oracle surface therefore keeps Betti/topology observations, spectral admissibility, identity hashes, and cochain commitments as separate public predicates.

This modular split is part of the mathematical statement. It prevents cryptographic assumptions, public classifiers, and Hodge-theoretic claims from being conflated. Exact and oracle invariants are used here as mathematical acceptance surfaces; their full CS/protocol treatment is deferred to the subsequent protocol paper.

5 Public Artifact Stack And Lossy Projection

Commitments and public shadows are the first concrete public-facing objects in the paper. They explain what public data can carry without revealing the hidden Hodge representative. They are the scaffolding for transparency in action and anonymity in identity: actions are exposed through auditable public artifacts, while identities remain inside hidden representatives and public equivalence classes.

5.1 Public, Artifact, And Private Layers

The public-facing stack has three mathematical levels. The fully public handle level contains hashes, roots, topology hashes, nullifiers, public summaries, and oracle judgments. The verifier-facing artifact level contains public shadows, cochain commitments, proof-carrying cochain payloads, and certificate

data. The private witness level contains the fibration representative, transport/restriction data, full cochains and openings, and the Hodge operator whose kernel selects the harmonic component. The paper's claims are stated so that public handles and verifier-facing artifacts bind to hidden witnesses without turning the hidden witness into public data. This is compatible with the sheaf-theoretic attacker-observation viewpoint of [Ino26]; the associated security games and runtime observation model belong to the subsequent protocol paper. The present paper uses only the mathematical public/private separation described below.

5.2 The Artifact Stack

The public artifact stack has the following shape:

hidden/private object
→ public shadow or compressed public datum
→ commitment/root/hash artifact
→ proof-carrying artifact
→ verifier judgment.

Each layer binds to or summarizes the preceding layer. A hash does not replace the mathematical object, a commitment does not replace the witness, and a proof artifact does not replace the proof obligation. The purpose of the stack is to let public verification refer to hidden sheaf/cochain data without exposing it. In this sense, the stack separates what must be publicly accountable from what must remain representative-private.

Hashes serve as public names, integrity bindings, and fixed-size pointers for larger artifacts. A topology hash names the public combinatorial support; a root commits to a batch or storage view; a transcript hash domain-separates a proof statement. None of these hashes asserts validity by itself. Validity comes from the predicate or proof attached to the hashed object.

5.3 Cochain Commitments

Cochain commitments provide the public handle on hidden cochain data. The Lean surface treats the canonical payload as a mathematical object with a modulus, edge values, declared edge count, and topology hash.

5.4 Commitment Map

Definition 5.1 (Cochain commitment payload)**.** *A cochain commitment payload is a tuple*

$$(q, n, c, \tau)$$

consisting of a modulus q, a declared edge count n, a list c of integer edge values, and a topology hash τ. In the canonical Lean payload this is `CochainCommitmentData`*.*

The intended commitment map is the transaction-facing coboundary map

$$(m, r) \longmapsto \delta(m\|r) \in C^1 \pmod{q},$$

stored together with the topology hash of the supporting complex. The payload therefore preserves the accepted modulus, the edge-vector shape, the public topology binding, and the serialized C^1-level commitment values. It does not expose the hidden fibration representative, private transport data, the operative Hodge–Laplacian, or the harmonic kernel.

Definition 5.2 (Commitment well-formedness)**.** *A cochain commitment payload is well formed when its modulus is the supported canonical modulus, its declared edge count agrees with the actual edge-value list length, and the edge list is nonempty.*

The security interface for this commitment layer consists of binding, hiding, homomorphic addition, homomorphic scalar multiplication, topology binding, and partial-opening soundness. The blinded-commitment, vector-commitment, and cohomological-MAC interfaces are separate axiom records rather than consequences silently bundled into the payload shape.

5.5 Commitment And Artifact Invariants

The commitment-facing invariants attach to the commitment payload and to the public artifact stack, not to the proximity projection. Payload well-formedness is a shape condition: it asserts the supported modulus, agreement between the declared edge count and the edge-value list, and a nonempty edge list. It is not a cryptographic uniqueness theorem and it does not assert that the cochain vector is nonzero. For example, the canonical zero payload is well formed whenever the declared edge count is positive.

Topology compatibility is a separate public invariant. Two committed cochains can be aggregated only when their moduli, declared edge counts, and topology hashes agree. The weaker same-topology relation records equality of

topology hashes alone. Cryptographic binding and topology-binding claims are supplied by the commitment-security axioms, not by the raw payload record.

At the artifact level, Lean records a hashed committed cochain as a public cochain payload together with digest data and a binding predicate. A proof-carrying committed cochain adds a proof system, a statement, and a verification predicate. The basic theorem exposed to this paper is that a well-formed proof-carrying committed-cochain artifact has an underlying well-formed cochain commitment payload.

5.6 Sheaf Storage And Witness Material

Storage roots, topology hashes, cochain roots, nullifiers, and proof objects are public artifacts in the stack. Full sheaf, cochain, and fibration data remain witness material unless a protocol path explicitly exposes them. The storage mechanics are protocol-level concerns; the mathematical point here is that stored public artifacts are shadows of accepted witness data, not replacements for the hidden object.

5.7 Lossy Projection

Compression creates verifier-facing public artifacts from hidden or full witness data. The compression need not be injective; it must preserve enough structure to support the relevant public statement.

A public shadow is allowed to forget hidden data. In fact, it must forget enough data that it does not select a private representative. The relevant question is not whether the public datum is injective, but which facts survive the compression relation.

5.8 Preserved And Lost Structure

The preserved structure before the finite-code layer is:

(i) public classifier or fingerprint equality for hidden representatives;

(ii) cochain commitment payload well-formedness;

(iii) topology binding for cochain commitments;

(iv) digest/root binding for compressed public shadows;

(v) proof-carrying validity of the public statement attached to an artifact.

The lost structure is the private representative itself: full sheaf/cochain witness data, hidden transport, the operative Hodge operator, private randomness, and runtime lifecycle facts. Those may be bound to public artifacts or certified by a proof-carrying statement, but they are not made public merely by the presence of a hash, root, or commitment payload.

5.9 Fundamental Properties

Proposition 5.3 (Well-formed committed payloads)**.** *A well-formed cochain commitment payload has nonempty edge data and modulus greater than one.*

Proposition 5.4 (Topology-compatible aggregation)**.** *Commitment aggregation is only meaningful for payloads with compatible modulus, declared edge count, and topology hash. This is the public topology-binding condition consumed by the chain layer.*

Proposition 5.5 (Proof-carrying artifact exposes payload)**.** *A well-formed proof-carrying committed-cochain artifact exposes a well-formed underlying cochain commitment payload.*

Byte parsing, allocation bounds, canonical encoding, concrete hash functions, domain separation, storage inclusion proofs, proof verification, state updates, and ephemeral-fiber lifecycle properties are TCB obligations. Lean records their boundary predicates; it does not identify them with the mathematical cochain commitment payload.

6 Reed–Solomon Projection And MCA

The concrete projection chain belongs here because its endpoint is the Reed–Solomon received-word surface and the mutual-correlated-agreement theorem. This section uses Grassmannian data in a different role from the HCP Grassmannian of Sections 2–4. Earlier, $W \in \mathrm{Gr}(k, R^n)$ parameterizes hidden harmonic kernels and projection Laplacians $\Delta_W = I - P_W$. Here, after the hidden/geometric object has already been accepted, a frame for the Grassmannian point is used as the source of a lossy proximity projection through Plucker coordinates and actual cochain commitments toward a finite Reed–Solomon target.

6.1 Projection Chain

The staged chain has the following shape:

FrameWitness
→ Projective Plucker shadow
→ committed cochain/local syndrome
→ plane shadow
→ finite-field received word
→ canonical Reed–Solomon target.

Each stage has local proximity gates and certificates. The orchestrating projection-chain theorem composes these local certificates without rechecking them. These are proximity-specific projection gates, not the main Homotopical Encrypt acceptance layer from Section 4. Their role is to certify the lossy path into the Reed–Solomon endpoint after the main hidden/geometric object has already been accepted. Thus the Plucker shadow is not part of the definition of HCP and is not the source of hidden-operator hardness. It is the first public projective encoding used by the proximity proof.

The semantic roles in this chain are separated. The private witness consists of the fibration data, frame choices, paths, and openings. The shadow is the Plucker/cochain/proximity data derived from that witness. A hash or root is only a compressed public handle to canonical bytes for the shadow, not the mathematical shadow itself. A certificate or SNARK proves that the private witness produced the accepted shadow and satisfied the stage invariants.

6.2 Projection-Stage Gates

The source and Plucker stages carry metric and public-shadow invariants. We use the usual Grassmannian and Plucker-coordinate algebra: a frame $(w_1, \dots, w_r)$ determines the exterior coordinate $[w_1 \wedge \cdots \wedge w_r] \in \mathbb{P}(\bigwedge^r V)$, with the quadratic Plucker relations as the projective Grassmannian equations [GH78]. The source disagreement score $\rho(W)$ is required to be nonnegative and invariant under the public Plucker quotient. The Plucker metric is defined only on the admissible projective chart, and its distance must be nonnegative and frame-quotient invariant. Public-shadow respect says that equality of the public Plucker shadow determines the same committed cochain, plane shadow, and received word along this projection path.

The Plucker-to-cochain stage carries the local-syndrome interface and consumes the actual cochain commitment layer of Section 5. Its target is not an abstract stand-in for a cochain; it is the existing committed payload

$$(q, n, c, \tau)$$

with supported modulus, declared edge count, edge values, and topology hash. A concrete encoder may send valid Plucker frame data to such a well-formed payload with the expected edge count, but the storage-native statement is relational: an observed or proof-carrying cochain commitment c realizes the public Plucker shadow. This is the boundary formalized by the public-binding surface: the chain/storage layer accepts the actual commitment payload, while the proximity proof supplies the statement that this payload realizes the Plucker shadow.

Syndrome adequacy then states that a well-formed committed cochain c with vanishing local syndrome $\sigma(c) = 0$ is a global-section candidate for the chosen sheaf. In the concrete bridge, the local syndrome is the commitment-sum witness; global candidates have zero commitment sum. The point is that the proximity proof passes through real cochain commitments and their well-formedness/topology-binding interface, not through a separate synthetic cochain carrier.

The cochain-to-plane and plane-to-RS stages carry semantic-alignment and non-collapse conditions. Semantic alignment requires valid frames, the plane shadow, and the received word to agree with the extraction maps. Plane extraction records the cochain well-formedness, expected edge count, and selected-index bounds needed to form the two-coordinate shadow. The far-region conditions assert calibrated non-collapse from the Plucker source region, through the committed cochain, to the final Reed–Solomon received word.

6.3 Canonical Reed–Solomon Endpoint

The proximity endpoint is stated over the paper-style alphabet shape $C \subseteq (\mathbb{F}^s)^n$. The canonical Homotopical Encrypt evaluation is the ordinary Reed–Solomon slice at width $s = 1$, using the standard polynomial evaluation-code viewpoint of Reed–Solomon codes [RS60]. The concrete endpoint used by the Lean surface fixes

$$\mathbb{F} = \mathbb{Z}/12289\mathbb{Z}, \qquad n = 4096, \qquad k = 1024,$$

so the rate is $1/4$. The evaluation domain is a certified smooth power domain: the chosen generator has order 12288, and cubing it gives an element of order

4096. Hence the evaluation points are

$$1, \alpha, \alpha^2, \ldots, \alpha^{4095},$$

with $\alpha^{4096} = 1$ and no earlier repetition. The canonical radius used for the below-Johnson comparison is the absolute Hamming radius

$$2047.$$

Lean proves the field arithmetic, domain cardinality, smoothness, evaluation-point compatibility, and absolute-radius normalization for this endpoint; the deep list-decoding and MCA estimates enter only as cited code-theoretic theorem slots.

6.4 Mutual Correlated Agreement

Definition 6.1 (MCA bad-challenge fraction)**.** *Let $C \subseteq (\mathbb{F}^s)^n$ be a block code and let $\ell(\gamma) = f_1 + \gamma f_2$ be an affine line of received words. For a radius δ, the bad challenges are those $\gamma \in \mathbb{F}$ for which $\ell(\gamma)$ is δ-close to C on a large coordinate set, but the pair (f_1, f_2) is not correspondingly close to the two-fold interleaved code $C^{\oplus 2}$ on that same set. The MCA error is the fraction of bad challenges in $\mathbb{F}$.*

The Lean endpoint specializes this paper definition to the canonical scalar Reed–Solomon slice $s = 1$. The width-one comparison is formalized in `CanonicalRS.MCA.Concrete`: ordinary scalar RS words are identified with words in $(\mathbb{F}^1)^n$, and the bad-challenge predicate is shown to agree with the paper-alphabet predicate.

Theorem 6.2 (Canonical RS MCA bound)**.** *For the canonical Homotopical Encrypt Reed–Solomon endpoint at the below-Johnson radius, every projection-induced final affine line has bad-challenge fraction at most*

$$\frac{\tau}{|\mathbb{F}|},$$

where τ is the cited MCA threshold for the canonical endpoint.

The deep list-decoding and MCA estimates are not reproved in Lean. They enter through `CanonicalRS.CitedBounds` as named theorem constants: the Johnson list-decoding bound, the count and fractional MCA bounds, and the paper Definition 4.3 width-one MCA bound. Lean proves the endpoint plumbing, the $s = 1$ equivalence, and the specialization to projection-induced

lines. The list-decoding background is the usual Reed–Solomon list-decoding theory, for example Guruswami–Sudan and its standard formulations [GS99, Gur04]; the project-specific MCA input is the cited Proximity Prize theorem boundary [Pro26].

6.5 Corollary

Corollary 6.3 (Verifier-facing RS/MCA projection)**.** *Given an accepted hidden/geometric witness, a certified Plucker-to-cochain-to-plane-to-RS projection chain, the required hard projection-gate evidence, and a canonical Johnson-radius far-separation witness, the public verifier obtains a finite Reed–Solomon received word whose canonical MCA bad-challenge fraction is bounded by the cited threshold $\tau/|\mathbb{F}|$.*

This corollary is a proximity statement. It says that accepted hidden/geometric state can be projected to a finite public RS/MCA target with calibrated non-collapse at the selected far region. It does not say that the projection is injective, and it does not turn the RS/MCA endpoint into the source of HCP hardness.

7 Formal Verification

7.1 Lean Formalization

The Lean development records the mathematical predicates, reductions, projection stages, local certificates, and trusted theorem boundaries used by the paper. The proof is not a proof of the Rust runtime; runtime parsing, canonical encodings, domain separation, and proof verification are handled by a separate TCB boundary.

7.2 Repository Structure

The formalization is split by theorem surface.

(i) `Axioms` contains the foundation, fibration, sheaf, Heyting, oracle, commitment, spectral, and stage-admissibility interfaces.

(ii) `Laplacian`, `CVP`, and `Hardness` contain the projection-model Hodge facts, explicit-lattice CVP comparison, public projection ambiguity, and Hodge/CVP theorem surfaces.

(iii) `Gates` contains gate-facing aliases and lemmas for the exact, spectral, oracle, and stage invariants stated with the corresponding mathematical objects.

(iv) `Proximity` contains the Plucker shadow, cochain bridge, plane projection, Reed–Solomon endpoint, projection chain, and proximity-specific gate certificates.

(v) `Proximity.CanonicalRS` contains the canonical RS endpoint, bridge data, cited bounds, and MCA specialization.

(vi) `TCB` contains proof-carrying artifacts, compression boundaries, observation boundaries, SNARK specialization, witness verification, and runtime-owned obligations.

(vii) `Chain` contains the chain-level transaction, block, ledger, and acceptance predicates that consume these surfaces in the subsequent protocol paper's protocol setting.

7.3 Theorem Correspondence

The core correspondence is:

HCP hidden-operator vocabulary.
`Hardness . MatrixIsHarmonic`, `Hardness . PublicProjection`, `Hardness.HiddenOperatorFiber`, and `Hardness.KernelAmbiguous`; Lean-defined surface.

Hodge decomposition.
`Hardness.Hodge_Decomposition_Theorem`; Lean-proved theorem surface for harmonic/exact/coexact decomposition.

Ambiguity hierarchy.
`Hardness . kernelAmbiguous _ implies _ lossy`, `Hardness . kernelAmbiguous _ of _ publicCompatibleHodgeAmbiguity`, and `Hardness.Hodge_Ambiguity`; Lean-proved.

Hodge/CVP reduction.
`Hardness . exists _ matrix _ ker _ eq _ range`, `Hardness . Hodge _ Hardness_Reduction_Proof`, and `Hardness.Hodge_Projection_Is_CVP`; Lean-proved.

Known-lattice CVP oracle insufficiency.
`Hardness.no_public_solver_on_ambiguous_input` and `Hardness.CVP_Oracle_Insufficient_For_Public_Hodge_Solving`; Lean-proved hidden-operator separation.

Hardness fibration semantics.
`Hardness.copy_equivalent_thm` and `Hardness.FibrationCopy_is_Grothendieck`; Lean-proved categorical interface.

Exact structural preconditions.
`Gates.Exact` and `Gates.structural_preconditions_of_sheaf_exact_gates`; alias/proved interface.

Oracle/Heyting judgment surface.
`Axioms.OracleModality`; assumption/interface.

Cochain payload well-formedness.
`CochainCommitmentData.WellFormed`; Lean-proved predicates.

Proof-carrying artifact.
`ProofCarryingCommittedCochain.WellFormed`; TCB boundary.

Proximity gate certificates.
`Proximity.Gates`; stage-local interfaces.

Projection chain.
`ProjectionChain.Chain.toEvaluationProjection`; Lean-proved plumbing.

Canonical RS bridge data.
`CanonicalRS.BridgeData`; Lean-proved endpoint facts.

RS list/MCA bounds.
`CanonicalRS.CitedBounds`; cited external theorem boundary.

Canonical MCA corollary.
`CanonicalRS.MCA.projectionBadChallengeFraction_le`; Lean-proved from cited bounds.

The status labels are part of the verification claim. Lean-proved rows are formal consequences of the imported modules. Cited external theorem rows are trusted mathematical inputs from the coding-theory literature. TCB rows are runtime obligations about parsing, encoding, hashing, proof verification, and state transition. Assumption/interface rows are explicit

predicates whose implementations or external justifications are not silently folded into the core HCP theorem.

8 Future Directions

8.1 Cryptographic Protocol

The present paper stops at the mathematical and formalization boundary. The subsequent protocol paper should turn the commitment, oracle, proof-carrying, and proximity surfaces into explicit adversary games and executable verification paths. The relevant Lean surfaces are already separated: `TCB` records proof-carrying artifacts and runtime obligations, `Gates` records object-local admissibility predicates, and `Consensus` records the Proof-of-Value scoring and policy layer. The concrete protocol must specify which certificates are public evidence, which witnesses remain private, and which runtime failures cause rejection, hold, or re-evaluation.

8.2 Anonymous Identity System

The hidden-representative/public-classifier split suggests an anonymous identity architecture. The public identity surface would expose class values, hashes, and oracle judgments, while the private side would retain fibration representatives, transport data, and the Hodge operator selecting the operative kernel. In Lean, this direction is anchored by `Axioms.HomotopyClass`, `Axioms.Fibration`, and `Axioms.OracleModality`. A future system should make the unlinkability, class-membership, and nullifier properties explicit rather than treating public-shadow equality as strict identity.

8.3 Sheaf Storage

The commitment layer can also be developed as a storage and retrieval system for controlled sheaf/cochain projections. The public store would retain topology hashes, cochain roots, compressed shadows, and proof-carrying statements. Full sheaf and fibration witnesses would remain private or be opened only through dedicated proof paths. The relevant Lean interfaces are `Axioms.CochainCommitment`, `TCB.CompressionBoundary`, `TCB.ObservationBoundary`, and `Proximity.Public`.

8.4 Vertra Chain

Vertra is the working name for the chain/network layer. The paper should keep the mathematical protocol name separate from product and token naming. The chain layer belongs in the subsequent protocol paper: it consumes the mathematical surfaces from this paper as protocol evidence, but it also requires block validity, ledger updates, fork choice, oracle attestation, worker reliability, and runtime trust-boundary analysis. A protocol-facing reference point is the higher categorical cryptoeconomics program [SKdS25]; it is not used in the proofs of the present paper. Lean files such as `Chain`, `Consensus`, and `Consensus.ProofOfValue` already mark this boundary without making those architecture claims part of the HCP theorem.

8.5 Complexity-Theoretic Extensions

The current formalization already isolates the hidden-operator obstruction: public data does not canonically determine the operative Hodge–Laplacian kernel. It also separates the external known-lattice CVP assumption from the project-native hidden-operator ambiguity, gives public-data-only oracle insufficiency statements, records classical and quantum adversary surfaces, and proves infinitude results for the relevant Grassmannian/Laplacian families. Future work should build on those surfaces quantitatively: rates of growth for hidden-kernel families, leakage bounds for public classifiers, success probabilities for restricted adversary classes, and reductions between refined HCP variants.

A Repository Dependency Graph

The Lean formalization is organized so that each paper section has a small number of import surfaces.

HCP and CVP.
`Axioms/*` feeds `Laplacian/*`, `CVP`, and `Hardness`.

Commitments and public artifacts.
`Axioms.CochainCommitment/*` feeds `TCB/*` and `Proximity.CochainBridge`.

Projection chain.
`Proximity.GrassmannianPlucker` feeds `Proximity.PluckerCochain`, then `Proximity.CochainPlane`, then `Proximity.CochainReedSolomon`.

Projection certificates.
`Proximity.ProjectionChain` feeds `Proximity.Gates`, which feeds `Proximity.CanonicalRS`.

Protocol boundary.
`TCB/*`, `Gates/*`, `Consensus/*`, and `Chain` are surfaces for the subsequent protocol paper.

For Paper 1, the main Lean check targets are:

```
lakeenvleanAxioms.lean      lakeenvleanGates.lean
lakeenvleanTCB.lean         lakeenvleanHardness.lean
lakeenvleanProximity.lean
```

The full build target remains `lakebuild`; the smaller targets above are the theorem-surface checks most directly corresponding to this paper.

B Lean/Paper Correspondence

B.1 Definitions And Interfaces

HCP public observation and hidden witness.
Sections 2–3; `Hardness.PublicProjection`, `Hardness.HiddenOperatorFiber`, and `Hardness.HiddenOperatorClass`.

Matrix harmonicity and Hodge instances.
Section 2; `Hardness.Hodge_Instance` and `Hardness.MatrixIsHarmonic`.

Kernel ambiguity and lossy public data.
Sections 2–3; `Hardness.SameKernel`, `Hardness.KernelAmbiguous`, `Hardness.PublicCompatibleHodgeAmbiguity`, and `Hardness.LossyPublicProjection`.

Public-data-only Hodge solvers.
Section 3; `Hardness.PublicHodgeInstance`, `Hardness.PublicHodgeSolver`, `Hardness.PublicHodgeSolverSound`, and `Hardness.PublicHodgeSolverExactOnHarmonic`.

Explicit-lattice CVP comparison.
Section 3; `CVP.CVP_Instance`, `CVP.CVPSolution`, and `CVP.CVPOracleCorrect`.

Hodge/CVP solution predicates.
Section 3; `Hardness.HodgeCVPSolution` and `Hardness.ProjectionHodgeCVPSolution`.

Hodge projection model.
Sections 2–3; `Laplacian.harmonicProjection`, `Laplacian.residual`, and `Laplacian.IsHarmonic`.

Categorical fibration copy surface.
Section 4; `Hardness . IsCartesianMorphism`, `Hardness . GrothendieckFibration`, `Hardness . HomotopyEquivalent`, and `Hardness.FibrationCopy`.

Semantic/public equality surface.
Section 4; `Axioms . HelferFOL`, `Axioms . HomotopyClass`, and `Axioms . Fibration`.

Heyting/oracle public judgment.
Section 4; `Axioms.Heyting3`, `Axioms.OracleQuery`, `Axioms.OracleAnswer`, and `Axioms.OracleEta2`.

Cochain commitment payload.
Section 5; `Axioms . CochainCommitmentData`, `CochainCommitmentData . WellFormed`, and `CochainCommitmentData.AddCompatible`.

Commitment security interface.
Section 5; `Axioms.CochainCommitmentAxioms`.

Proof-carrying public artifact.
Section 5; `TCB . HashedCommittedCochain` and `TCB . ProofCarryingCommittedCochain`.

Projection chain.
Section 6; `ProjectionChain . Chain` and `ProjectionChain . Chain . toEvaluationProjection`.

Proximity-stage certificates.
Section 6; `Proximity . Gates . SourceMetricGate`, `Proximity . Gates . SyndromeAdequacyGate`, and `Proximity.Gates.HardGateCertificate`.

Canonical RS/MCA endpoint.
Section 6; `CanonicalRS . BridgeData`, `CanonicalRS . MCA . Line`, and `CanonicalRS.MCA.badChallengeFraction`.

B.2 Theorems And Status

Kernel ambiguity implies lossy public projection.
`Hardness.kernelAmbiguous_implies_lossy`; Lean-proved.

Public-compatible Hodge ambiguity gives kernel ambiguity.
`Hardness.kernelAmbiguous_of_publicCompatibleHodgeAmbiguity`; Lean-proved.

Concrete Hodge ambiguity exists in nontrivial dimension.
`Hardness . Hodge _ Ambiguity`, `Hardness . constantPublicProjection _ publicCompatibleHodgeAmbiguity`, `Hardness . constantPublicProjection _ kernelAmbiguous`, and `Hardness . constantPublicProjection_lossy`; Lean-proved.

Every explicit lattice image can be a Hodge kernel.
`Hardness.exists_matrix_ker_eq_range`; Lean-proved linear-algebra construction.

Kernel realization for explicit lattices.
`Hardness.Hodge_Hardness_Reduction` and `Hardness.Hodge_Hardness_Reduction_Proof`; Lean-proved surface.

Hodge decomposition into harmonic, exact, and coexact parts.
`Hardness.Hodge_Decomposition_Theorem`; Lean-proved theorem surface.

Ambiguous input contradicts public-data-only Hodge solving.
`Hardness.public_solver_eq_input_of_harmonic` and `Hardness.no_public_solver_on_ambiguous_input`; Lean-proved lemmas.

Known-lattice CVP does not select the hidden operator.
`Hardness.CVP_Oracle_Insufficient_For_Public_Hodge_Solving`; Lean-proved surface.

Strict hidden-operator separation.
`Hardness.Hodge_Strictly_Harder`; Lean-proved model theorem.

Projection-model Hodge recovery is CVP-shaped.
`Hardness.matrix_harmonic_iff_projection_isHarmonic`, `Hardness.Hodge_Projection_Is_CVP`, and `Hardness.harmonicProjection_Is_ProjectionHodgeCVPSolution`; Lean-proved.

Fibration copies preserve the hidden semantic surface.
`Hardness.copy_equivalent_thm` and `Hardness.FibrationCopy_is_Grothendieck`; Lean-proved categorical interface.

Exact sheaf gates supply $d^2 = 0$ and Laplacian-from-coboundary.
`Gates.structural_preconditions_of_sheaf_exact_gates`; Lean-proved alias/interface.

Well-formed payloads have nonempty edge data and supported modulus.
`CochainCommitmentData.edge_values_nonempty_of_wellFormed` and `CochainCommitmentData.modulus_gt_one_of_wellFormed`; Lean-proved.

Proof-carrying artifacts expose the committed payload.
`TCB.proofCarrying_cochain_wellFormed`; TCB boundary theorem.

Projection chain composes to one evaluation projection.
`ProjectionChain.Chain.toEvaluationProjection`; Lean-proved plumbing.

Syndrome adequacy turns zero local syndrome into codeword/global candidate.
`Proximity.Gates.SyndromeAdequacyGate.codeword_of_wellFormed_zeroSyndrome`; stage-local interface.

Final proximity evidence yields calibrated end-to-end readiness.
`Proximity.Gates.CalibratedFinalCertificate.toCalibratedEndToEndReady`; Lean-proved plumbing.

Canonical RS list/MCA bounds enter at a cited boundary.
`CanonicalRS.CitedBounds.bounds`; cited external theorem package.

Canonical MCA bad-challenge fraction is bounded after projection.
`CanonicalRS.MCA.projectionBadChallengeFraction_le`; Lean-proved from cited bounds.

C Complete Lean Coverage Ledger

This ledger is the audit trail for the translation. The body of the paper gives the mathematical definitions and proofs; this appendix records where each Lean declaration family enters the written account. Import-only aggregate files are listed with the files they collect.

C.1 Build And Aggregate Files

`lakefile.lean`.
Build configuration; no mathematical declaration is translated as a theorem.

`Axioms.lean`, `Gates.lean`, `TCB.lean`, `Consensus.lean`, `Proximity.lean`, and `Proximity/CanonicalRS.lean`.
Aggregate import surfaces. Their mathematical content is exactly the union of the component files listed below.

C.2 HCP, CVP, And Fibrational Algebra

`Hardness.lean`.
Sections 2–4 translate every declaration into the hidden Hodge-operator algebra. The HCP data are `Hodge_Instance`, `MatrixIsHarmonic`, `PublicProjection`, `PubliclyEquivalent`, `HiddenOperatorFiber`, and `HiddenOperatorClass`. The ambiguity predicates are `LossyPublicProjection`, `SameKernel`, `KernelAmbiguous`, and `PublicCompatibleHodgeAmbiguity`. The elementary closure lemmas `mem_hiddenOperatorClass_self`, `sameKernel_refl`, `kernelAmbiguous_implies_lossy`, and `kernelAmbiguous_of_publicCompatibleHodgeAmbiguity` are Theorem 2.10. The concrete ambiguity theorems `Hodge_Ambiguity`, `Hodge_Ambiguity_Proof`, `constantPublicProjection_publicCompatibleHodgeAmbiguity`, `constantPublicProjection_kernelAmbiguous`, and `constantPublicProjection_lossy` are Theorem 3.4. The known-lattice comparison declarations `Hodge_Solver_Correct`, `exists_matrix_ker_eq_range`, `Hodge_Hardness_Reduction`, and

Hodge_Hardness_Reduction_Proof are Theorem 3.1. The decomposition theorem Hodge_Decomposition_Theorem is Theorem 2.5. The public-solver predicates PublicHodgeInstance, PublicHodgeSolver, PublicHodgeSolverSound, PublicHodgeSolverExactOnHarmonic, public_solver_eq_input_of_harmonic, no_public_solver_on_ambiguous_input, Hodge_Strictly_Harder, and CVP_Oracle_Insufficient_For_Public_Hodge_Solving are Theorem 3.3. The projection/CVP predicates HodgeCVPSolution, ProjectionHodgeCVPSolution, matrix_harmonic_iff_projection_isHarmonic, Hodge_Projection_Is_CVP, and harmonicProjection_Is_ProjectionHodgeCVPSolution are Theorem 3.2. The categorical declarations IsCartesianMorphism, GrothendieckFibration, HomotopyEquivalent, Wrapper, Fibration, FibrationCopy, ToWrapper, FromWrapper, copy_equivalent_thm, and FibrationCopy_is_Grothendieck are Theorem 4.1.

CVP.lean.
Section 3 translates ExplicitLattice, CVP_Instance, InExplicitLattice, IsClosestLatticeVector, CVPSolution, CVP_Oracle, LatticeOracleCorrect, CVPOracleCorrect, CVPSolverCorrect, cvpSolverCorrect_iff_oracleCorrect, CVPHardnessAssumption, and zero_lattice_oracle_correct as the known-lattice comparison surface.

Axioms / HelferFOL . lean, Axioms / Fibration . lean, and Axioms/HomotopyClass.lean.
Section 4 translates path equality, public classifiers, public fibers, class-membership witnesses, transport, fiber systems, ephemeral fiber specifications, and nonfaithful classifiers. The equivalence, symmetry, transitivity, membership, and nonfaithfulness lemmas justify the public fiber language used in HCP.

C.3 Foundational, Sheaf, Spectral, And Judgment Axioms

Axioms/Foundation.lean and Gates/Exact.lean.
Section 4 translates Foundation and FoundationWellFormed: finite cochain data, coboundary compatibility, Hodge Laplacian formation, topology connectivity, spectral gap positivity, and public key-image/freshness premises. The exact-gate aliases ExactGateBundle, SheafExactGateBundle, FoundationWithSheafExactGates, and structural_preconditions_of_sheaf_exact_gates are Theorem 4.2.

Axioms/SheafCoherence.lean.
Section 4 translates restriction-shape compatibility, edge naturality, triangle-face commutativity, sheaf cochain complexes, Hodge coherence, finite simplicial nerves, SheafCoherence, SheafCoherenceWellFormed, FoundationWithSheafCoherence, and coherence_implies_structural_preconditions.

Axioms/Grassmannian.lean.
Sections 4 and 6 translate the Grassmannian metric and geometry objects: `GrassmannianDistance`, `chordal_leq_geodesic`, `KarcherMean`, `SheafLaplacian`, `CechNerve`, `PluckerEligibility`, `ConsensusGeometry`, and `ConsensusGeometryWellFormed`.

Axioms/SpectralBounds.lean and Gates/Spectral.lean.
Section 4 translates the constants $\sqrt{2}/2$ and $2\sqrt{2}$, the predicates `SpectralGapInPoVBounds`, `NearPoVSpectralBoundary`, `TsirelsonPredicate`, `NearTsirelsonBoundary`, the aliases `SpectralGapGate` and `NearSpectralBoundary`, the geometry witness `ByConstructionPoVGeometry`, and the theorems `byConstruction_gap_in_bounds` and `byConstruction_passes_spectral_gate`.

Tsirelson.lean.
Section 4 translates the Grassmannian/Laplacian family `Grassmannian_Laplacians`, `ℍℙ_G`, `LaplacianSpace`, `SpectralGap`, `ℍℙ_1`, `projector_map`, `Classical_Strategy`, `Quantum_Strategy`, and `MaxEigenvalue`; the infinitude and injectivity results `R_infinite`, `infinite_lines`, `Grassmannian_Laplacian_injective`, `projector_injective`, `laplacian_injective`, `ℍℙ_1_infinite`, `R_uncountable`, `ℍℙ_1_infinite_v2`, and `ℍℙ_1_Nonempty`; the Laplacian-space inclusions `laplacian_properties`, `laplacian_is_hermitian`, `ℍℙ_1_subset_LaplacianSpace`, `ℍℙ_1_subset_LaplacianSpace_v3`, `ℍℙ_1_subset_LaplacianSpace_v5`, `ℍℙ_1_subset_LaplacianSpace_explicit_v2`, and `ℍℙ_1_subset_LaplacianSpace_explicit`; the classical/quantum statement forms `Classical_Bound_Statement` and `Quantum_Bound_Statement`; and the empty-family sanity checks `Classical_Bound_empty_family` and `Quantum_Bound_empty_family`.

Axioms/Heyting.lean and Axioms/OracleModality.lean.
Section 4 translates the three-valued Heyting algebra, meet, join, implication, negation, failure of double-negation elimination, abstract Heyting collapse, nonfaithful public truth, spectral judgments, consensus actions, oracle queries and answers, Ω_{nv}, witness collapses, quotient equivalence, sheafification, oracle forcing, η_1/η_2 oracle modalities, Lawvere–Tierney idempotence, boundary judgment behavior, canonical consensus admissibility, oracle pipelines, and the oracle/consensus bridge theorems.

Axioms / StageAdmissibility . lean, Gates / Stage . lean, and Gates/Oracle.lean.
Section 7 records the stage and oracle aliases: pipeline stages, stage admissibility, transaction-critical invariant bundles, Python-stage snapshots, refinement predicates, and the theorem that a refined transaction-critical pass is admissible.

C.4 Laplacian Geometry And Search Spaces

`Laplacian/Hodge.lean` and `Laplacian.lean`.
Sections 2–3 translate harmonic projection, residual vector, residual norm, harmonicity, projection membership, nonnegativity, and all equivalences between zero residual, membership in the harmonic subspace, and harmonicity.

`Laplacian/Projection.lean`.
Section 3 translates the projection Laplacian $\Delta_W = I - P_W$ and its pointwise application formula.

`Laplacian/Spectral.lean`.
Section 4 translates sheaf Laplacians, spectral fingerprints, spectral equivalence, scalar Laplacians, kernel and eigenvalue lemmas, scalar fingerprints, homotopy-equivalent Laplacians, and the theorem that spectrum is not homotopy invariant.

`Laplacian/Likelihood.lean`.
Section 4 records the classical and quantum adversary shapes, valid quantum adversaries, Gaussian likelihoods, minimum spectral gaps, normalized success probabilities, optimal-adversary likelihood, and the Gaussian factorization lemmas as spectral-statistical surfaces rather than HCP hardness assumptions.

`Laplacian/Grassmannian.lean`, `Laplacian/Grassmannian/Injectivity.lean`, `Laplacian / Grassmannian / SearchSpace . lean`, and `Laplacian/Grassmannian/ParametricFamily.lean`.
Sections 4 and 6 translate harmonic subspaces, the map to projection Laplacians, the families Ω_r and Ω_1, membership in Laplacian space, nonemptiness of Ω_k, injectivity of the Grassmannian Laplacian map, and the parametric construction of families of k-planes.

C.5 Commitments, Public Artifacts, And TCB Boundary

`Axioms/CochainCommitment/Payload.lean`, `Axioms/CochainCommitment/Security.lean`, and `Axioms/CochainCommitment.lean`.
Section 5 translates the canonical modulus, byte lengths, wire length, `CochainCommitmentData`, edge-count matching, nonempty edge data, supported modulus, well-formedness, commitment sums, common-topology and additive compatibility, zero and addition operations, payload lemmas, binding/hiding/additivity/security axioms, blinded and vector commitment axioms, cohomological MAC axioms, and the canonical commitment construction from foundational data.

`TCB / ProofCarrying . lean`, `TCB / Runtime . lean`, `TCB / SNARK . lean`, `TCB / CompressionBoundary . lean`, `TCB / ObservationBoundary . lean`, and `TCB/WitnessVerifier.lean`.
Sections 5 and 7 translate hash words, hashed committed cochains, proof-carrying cochains, runtime specification boundaries, runtime acceptance

predicates, SNARK-carrying cochains, compression contracts, compressed shadows, observation boundaries, gap-decision certificates, witness-compression envelopes, public state updates, and theorems stating that accepted artifacts expose well-formed cochain and oracle predicates.

`Axioms/ZKP.lean`.
Section 5 records the zero-knowledge proof object and its well-formedness predicate as an assumed proof-carrying artifact interface.

C.6 Projection Chain, Proximity Gates, And RS/MCA Endpoint

`Proximity / Problem . lean`, `Proximity / Public . lean`, `Proximity/Projection.lean`, and `Proximity/PluckerShadow.lean`.
Section 6 records the problem-facing public projection vocabulary that prepares the Plucker and finite-code shadow.

`Proximity/Grassmannian.lean`, `Proximity/GrassmannianPlucker.lean`, and `Proximity/GrassmannianPlucker/Concrete.lean`.
Section 6 translates harmonic Grassmannian points, exterior coordinates, frame witnesses, decomposability, framed systems, raw and public shadows, soundness, projective equivalence, projective shadows, projective normalization, projective framed systems, and projective sound inputs.

`Proximity / CochainBridge . lean`, `Proximity / PluckerCochain . lean`, `Proximity / PluckerCochain / Concrete . lean`, and `Proximity/PluckerCochain/PublicBinding.lean`.
Section 6 translates valid frames, Plucker-to-cochain instances, local syndrome targets, codeword predicates, evaluation projections, invariant checklists, payload encoders, commitment-sum residuals, payload realizations, hashed Plucker shadows, observed frame bindings, and the theorems that valid public shadows bind to well-formed cochains and local codewords.

`Proximity / CochainPlane . lean`, `Proximity / CochainPlane / Core . lean`, `Proximity/CochainPlane/Concrete.lean`, `Proximity/CochainPlane/ReedSolomon.lean`, and `Proximity/CochainReedSolomon.lean`.
Section 6 translates plane shadows, plane extraction contexts, local non-degeneracy, alignment, projected planes, plane targets, plane-to-evaluation projections, local-codeword-to-target-codeword theorems, distance monotonicity, canonical-field plane encodings, interpretations, and RS received words.

`Proximity/ProjectionChain.lean` and `Proximity/ProjectionChain/Core.lean`.
Section 6 translates the complete projection chain, including accessors for committed cochain, plane shadow, received word, and the composite evaluation projection.

`Proximity / Gates . lean`, `Proximity / Gates / Stage . lean`, `Proximity/Gates/Final.lean`, and `Proximity/GateEvidence.lean`.
Section 6 translates only proximity gates: source metric, Plucker metric, semantic alignment, syndrome adequacy, public-shadow respect, noncollapse, far-region separation, runtime binding, structural bundles, hard-gate certificates, calibrated final certificates, certified state buses, stage checklists, stage certificates, radius calibrations, final readiness, evidence slots, evidence databases, and the binding/completeness lemmas for proximity evidence.

`Proximity / RS / ListDecoding . lean`, `Proximity / RS / ReedSolomon . lean`, `Proximity/RS/CorrelatedAgreement.lean`, `Proximity/RS/Target.lean`, and `Proximity/RS.lean`.
Section 6 translates words, block codes, Hamming and relative Hamming balls, list-size bounds, interleaved and additive codes, RS parameters, vector codewords, correlated-agreement and mutual-correlated-agreement predicates, bad-parameter sets, bridge obligations, calibrated $HomotopicalEncrypt$-to-RS bridges, and pullback-correctness theorems.

`Proximity/CanonicalRS/Parameters.lean`, `Proximity/CanonicalRS/PowerDomain.lean`, `Proximity/CanonicalRS/Encoding.lean`, `Proximity/CanonicalRS/Metric.lean`, `Proximity/CanonicalRS/Target.lean`, `Proximity/CanonicalRS/FarSeparation.lean`, `Proximity / CanonicalRS / BridgeData . lean`, `Proximity / CanonicalRS / BridgeWitness . lean`, `Proximity / CanonicalRS / BridgeCertificate . lean`, `Proximity / CanonicalRS / CitedBounds . lean`, `Proximity/CanonicalRS/MCA.lean`, `Proximity/CanonicalRS/MCA/Line.lean`, and `Proximity/CanonicalRS/MCA/Concrete.lean`.
Section 6 translates the canonical finite field, length, degree bound, rate, Johnson and capacity radii, smooth domains, power-root domain, canonical encoding, Hamming-to-code metrics, target bounds, far-separation witnesses, bridge data and theorem slots, bridge certificates, cited BCHKS-style bounds, MCA block code, challenge and bad-challenge fractions, and the bound on the projected bad-challenge fraction.

C.7 Sequel-Facing Protocol And Implementation Boundary

`Axioms/Consensus.lean`, `Axioms/Ledger.lean`, `Axioms/Coregulator.lean`, and `Axioms/PythonHomotopyRefinement.lean`.
Section 7 records consensus, ledger, coregulator, and Python homotopy refinement as boundary objects. Their full protocol semantics belong to Paper 2, but their predicates are named here so the Paper 1 mathematical objects have explicit downstream interfaces.

`Consensus / Geometry . lean`, `Consensus / Reliability . lean`, and `Consensus/ProofOfValue.lean`.
Section 7 records Plucker eligibility and consensus geometry aliases, reliability evidence and bounded reliability scores, proximity quality, Proof-of-Value

gates, Proof-of-Value inputs, scores with and without reliability/proximity, and the score bounds. These are protocol-facing uses of the Section 4 and Section 6 mathematical predicates.

`Chain.lean`.
Section 7 records validators, transactions, blocks, chains, spectral bounds, topology and freshness predicates, block validity, tips, heights, extension, transaction and block acceptance, ledger-state updates, key-image preservation theorems, validator eligibility, cumulative gap, preference, transaction composition and identities, quadratic weight, Sybil-resistance weighting, chain-growth monotonicity, fork-resolution transitivity, and connectedness of eligible validators. The detailed chain architecture remains Paper 2.

References

[AB26] Danel Ahman and Andrej Bauer. Sheaves as oracle computations. *arXiv preprint*, 2026.

[GH78] Phillip Griffiths and Joseph Harris. *Principles of Algebraic Geometry*. John Wiley & Sons, 1978.

[GS99] Venkatesan Guruswami and Madhu Sudan. Improved decoding of reed–solomon and algebraic-geometric codes. *IEEE Transactions on Information Theory*, 45(6):1757–1767, 1999.

[Gur04] Venkatesan Guruswami. *List Decoding of Error-Correcting Codes*, volume 3282 of *Lecture Notes in Computer Science*. Springer, 2004.

[Hel19] Adam Helfer. First-order homotopical logic. *arXiv preprint*, 2019.

[HG18] Jakob Hansen and Robert Ghrist. Toward a spectral theory of cellular sheaves. *arXiv preprint*, 2018.

[Ino26] Takao Inoué. Grothendieck topologies and sheaf-theoretic foundations of cryptographic security: Attacker models and Σ-protocols as the first step. *arXiv preprint*, 2026.

[LM94] Saunders Mac Lane and Ieke Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Universitext. Springer, 1994.

[Pro26] Proximity Prize authors. Open problems in list decoding and correlated agreement, 2026. Working citation placeholder; replace with full author metadata before submission.

[RS60] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.

[SKdS25] Melanie Swan, Takashi Kido, and Renato P. dos Santos. Higher categorical cryptoeconomics: HoTT blockchains, petri net computad ledgers, coregulator DAOs, and cohomology ZKPs, 2025. Working citation; author affiliations listed in project notes as University College London, Teikyo University, and academicum.ai. Replace with venue metadata before submission.

[Tsi80] Boris S. Tsirelson. Quantum generalizations of bell's inequality. *Letters in Mathematical Physics*, 4:93–100, 1980.